

NeoEng D-Core

Guia de uso, integração, diagnóstico e evidências

Baseline: v1.14.1 · Release publicada · Manual didático para engenharia e desenvolvimento

Este PDF foi gerado a partir do manual Markdown do repositório. As figuras de console são saídas reais de binários locais já construídos, executados em ambiente Windows local/simulado. Elas ilustram o uso; não constituem qualificação universal de hardware.

NeoEng D-Core — Guia de uso e integração

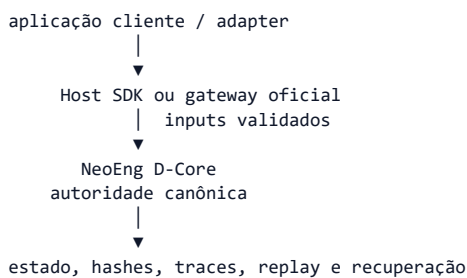
Versão do guia: 1.0 Baseline documentada: NeoEng D-Core v1.14.1 Público: engenheiros de integração, desenvolvedores de adapters, equipes de QA, suporte e auditoria técnica.

Este guia descreve o comportamento comprovado do produto. Ele não amplia os claims públicos, não transforma resultados de laboratório em garantia universal e não substitui os contratos normativos do repositório.

1. O que o NeoEng D-Core faz

O NeoEng D-Core é um núcleo C++23 horizontal que mantém uma autoridade canônica de estado. Um host fornece um estado inicial e inputs validados; o núcleo aplica a transição oficial e pode devolver estado, hashes, evidências, traces, checkpoints e resultados de recuperação.

TEXT



A dependência é unidirecional: o host consome o núcleo, mas não recebe ponteiro mutável para `WorldState`, `Body` ou snapshots internos.

O núcleo não é, por si só:

- um renderer ou uma UI;
- um anti-cheat;
- um transporte remoto de produção;
- um adapter pronto para Unity, Unreal, ROS ou um setor específico;
- um provedor de PKI, HSM, TPM ou assinatura assimétrica;
- uma certificação de hardware, desempenho ou segurança externa.

As claims autorizadas estão em `docs/commercial/PUBLIC_CLAIMS.md` e `audit/PRODUCT_CLAIMS_LEDGER.json`.

2. Regras fundamentais de uso

1. O `WorldState` canônico só é alterado pelas APIs oficiais. 2. Inputs de rede não devem ser enviados diretamente à ABI C. Eles devem passar pelo gateway autenticado e pelo adapter homologado. 3. O handle do Host SDK pertence à thread que o criou. Todas as operações, inclusive destruição, devem ocorrer nessa thread. 4. O host não deve depender de relógio de parede, tempo de disco, resposta de API externa ou RNG não controlado para decidir o estado canônico. 5. Um rollback só pode usar frames ainda retidos pela configuração. 6. `monotonic_time_ns` é diagnóstico operacional; ele não entra no hash canônico. 7. Um hash confirma identidade do estado observado; ele não prova que a regra de negócio esteja conceitualmente correta. 8. Toda medição deve ser registrada com commit, configuração, compilador, hardware, política de energia e saída bruta.

3. Construir e executar a suíte

Windows

Abra um Developer PowerShell do Visual Studio:

POWERSHELL

```
PS C:/NeoEng> Set-ExecutionPolicy -Scope Process Bypass
PS C:/NeoEng> cmake --preset windows-clang-release
PS C:/NeoEng> cmake --build --preset windows-clang-release --parallel 2
PS C:/NeoEng> ctest --preset windows-clang-release --output-on-failure
```

O preset pressupõe clang-cl, Ninja e VCPKG_ROOT configurados. O script operacional equivalente é:

POWERSHELL

```
PS C:/NeoEng> .\scripts\windows\run-dcore.ps1 -BootstrapDependencies -FullTestSuite
```

Linux

BASH

```
$ cmake --preset linux-gcc-release
$ cmake --build --preset linux-gcc-release --parallel
$ ctest --preset linux-gcc-release --output-on-failure
```

O resultado deve ser guardado junto com o commit, sistema operacional, compilador e configuração. Uma suíte aprovada confirma o escopo executado; não qualifica automaticamente ARM64, P0–P4 ou desempenho universal.

Opções relevantes

Opção	Finalidade
NEOENG_DCORE_BUILD_HOST_SDK	Compila e instala o Host SDK
NEOENG_DCORE_BUILD_DISTRIBUTED_REFERENCE	Compila a referência de duas instâncias
NEOENG_DCORE_BUILD_VIEW_LAB	Compila o diagnóstico visual somente leitura
NEOENG_ENABLE_SANITIZERS	Habilita ASan/UBSan nas campanhas compatíveis
NEOENG_ENABLE_LIBFUZZER	Habilita os alvos libFuzzer
NEOENG_WARNINGS_AS_ERRORS	Converte warnings em falhas de build

4. Instalação do Host SDK

Depois de instalar o pacote em um prefixo limpo, um consumidor CMake usa os targets exportados:

CMAKE

```
cmake_minimum_required(VERSION 3.25)
project(example_host LANGUAGES C)

find_package(NeoEngDCore CONFIG REQUIRED)

add_executable(example_host main.c)
target_link_libraries(example_host PRIVATE NeoEng::DCoreHostSdk)
```

O contrato completo está em docs/contracts/HOST_SDK_C_ABI_V1.md. A distribuição v1.14.1 documenta uma biblioteca estática; não há claim de distribuição universal como shared library.

5. Exemplo mínimo em C

O exemplo abaixo usa somente símbolos públicos do header modules/host_sdk/include/neoeng/dcore_host.h. Ele cria um corpo, avança um frame, obtém os hashes e destrói o handle.

C

```
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>
#include "neoeng/dcore_host.h"
```

```

static int require_ok(neoeng_dcore_status status, const char *operation) {
    if (status == NEOENG_DCORE_STATUS_OK) return 1;
    fprintf(stderr, "%s: %s (%" PRIu32 ")\\n",
        operation, neoeng_dcore_status_string(status), status);
    return 0;
}

static void print_hex32(const uint8_t *bytes) {
    for (uint32_t i = 0; i < NEOENG_DCORE_SHA256_BYTES; ++i)
        printf("%02x", bytes[i]);
}

int main(void) {
    neoeng_dcore_version_info version = {0};
    version.struct_size = (uint32_t)sizeof(version);
    if (!require_ok(neoeng_dcore_host_get_version(&version), "get_version"))
        return 1;

    neoeng_dcore_host_config config = {0};
    config.struct_size = (uint32_t)sizeof(config);
    if (!require_ok(neoeng_dcore_host_default_config(&config), "default_config"))
        return 1;

    /* Q32.32: 1 unidade inteira = 2^32 na representação raw. */
    const neoeng_dcore_body body = {
        1U, 0U, 0, 0, 0, 0
    };
    const neoeng_dcore_input input = {
        1U, 0U, INT64_C(1) << 32, 0
    };

    neoeng_dcore_host *host = NULL;
    if (!require_ok(neoeng_dcore_host_create(
        0U, &body, 1U, &config, &host), "create"))
        return 1;

    neoeng_dcore_state_summary summary = {0};
    summary.struct_size = (uint32_t)sizeof(summary);
    const neoeng_dcore_status advanced = neoeng_dcore_host_advance(
        host, &input, 1U, 1001U, 1000000U, &summary);
    if (!require_ok(advanced, "advance")) {
        neoeng_dcore_host_destroy(host);
        return 1;
    }

    printf("frame=%" PRIu64 " bodies=%" PRIu64
        " stable_hash=0x%016" PRIx64 "\\n",
        summary.frame, summary.body_count, summary.stable_hash);
    printf("canonical_sha256=");
    print_hex32(summary.canonical_sha256);
    printf("\\nmerkle_root_sha256=");
    print_hex32(summary.merkle_root_sha256);
    printf("\\n");

    if (!require_ok(neoeng_dcore_host_destroy(host), "destroy"))
        return 1;
    return 0;
}

```

O que o exemplo demonstra

- struct_size é informado antes de usar uma estrutura pública.
- O corpo e o input usam identificador não nulo.
- A aceleração é fornecida em Q32.32.
- correlation_id e monotonic_time_ns são identificadores/diagnósticos operacionais.
- O resultado inclui frame, quantidade de corpos e três identificadores de evidência.

O exemplo não prova desempenho, equivalência entre arquiteturas ou correção da regra de negócio. Ele demonstra o contrato básico da ABI.

6. Avanço, rollback e checkpoint

Avançar o estado

Use `neoeng_dcore_host_advance()` somente depois de validar o lote de inputs. O array é emprestado apenas durante a chamada; o caller continua responsável pelo armazenamento.

Corrigir um input histórico

Quando o frame ainda estiver retido, use:

C

```
uint64_t resimulated_frames = 0;
neoeng_dcore_state_summary corrected = {0};
corrected.struct_size = (uint32_t)sizeof(corrected);

neoeng_dcore_status status =
    neoeng_dcore_host_correct_input_and_resimulate(
        host,
        input_frame,
        corrected_inputs,
        corrected_input_count,
        correlation_id,
        monotonic_time_ns,
        &resimulated_frames,
        &corrected);
```

O frame precisa existir na janela de retenção. Se não existir, o resultado esperado é `NEOENG_DCORE_STATUS_NOT_FOUND` ou uma decisão equivalente de retenção conforme o caminho chamado. Não aumente silenciosamente a janela para esconder um erro de contrato.

Restaurar checkpoint

C

```
neoeng_dcore_state_summary restored = {0};
restored.struct_size = (uint32_t)sizeof(restored);
neoeng_dcore_status status = neoeng_dcore_host_restore_checkpoint(
    host, checkpoint_frame, correlation_id, monotonic_time_ns, &restored);
```

Depois de restaurar, registre o frame, o hash, o motivo e a nova linha temporal. O rollback não desfaz efeitos externos já confirmados.

7. Recuperação e falhas

O host pode relatar falhas externas controladas:

C

```
neoeng_dcore_recovery_event event = {0};
event.struct_size = (uint32_t)sizeof(event);

neoeng_dcore_status status = neoeng_dcore_host_report_fault(
    host,
    NEOENG_DCORE_FAULT_NETWORK_UNAVAILABLE,
    correlation_id,
    monotonic_time_ns,
    &event);
```

Leia `directive`, `action`, `mode`, `generation` e `rollback_checkpoint_frame`. Se `acknowledgement_required` estiver ativo, confirme somente a recuperação realmente realizada:

C

```
neoeng_dcore_recovery_ack_result result = {0};
result.struct_size = (uint32_t)sizeof(result);

status = neoeng_dcore_host_acknowledge_recovery(
    host,
    event.generation,
```

```

NEOENG_DCORE_RECOVERY_ACK_RETRY_LATER,
correlation_id,
0U,
monotonic_time_ns,
&result);

/* status pode ser OK mesmo quando result.accepted == 0. */
if (status == NEOENG_DCORE_STATUS_OK && result.accepted == 0U)
    fprintf(stderr, "recovery rejeitada: motivo=%" PRIu32 " "\n",
            result.reject_reason);

```

Nunca reutilize uma geração antiga. Os estados de recuperação rejeitam acknowledgements obsoletos, tipos incorretos e checkpoints incompatíveis.

8. Erros frequentes e correções

Status/sintoma	Causa provável	Correção
ABI_MISMATCH	Major diferente ou pacote/header incompatível	Verifique <code>get_version</code> , pacote instalado e <code>NEOENG_DCORE_HOST_ABI_MAJOR</code> .
WRONG_THREAD	O handle foi usado por thread diferente da criadora	Centralize o handle em uma thread determinística ou serialize o adapter.
INVALID_ARGUMENT	Ponteiro, <code>struct_size</code> , ID ou limite inválido	Inicialize estruturas, use IDs não nulos e valide contagens.
INVALID_STATE	Estado inicial ou input fora do contrato	Preserve invariantes e não injete estado mutável diretamente.
NOT_FOUND	Frame ou checkpoint expirou da janela	Reduza atraso, ajuste retenção conscientemente e registre a perda.
BUFFER_TOO_SMALL	Buffer de saída insuficiente	Faça consulta inicial com capacidade zero, leia <code>out_required_count</code> , aloque e repita.
RECOVERY_REQUIRED	O runtime entrou em uma fronteira de recuperação	Leia o evento pendente e siga a diretiva antes de avançar.
OUT_OF_MEMORY	Capacidade configurada ou workload excedido	Reduza limites, aplique backpressure e trate a falha; não faça wraparound.
NUMERIC_OVERFLOW	Operação Q32.32 não representável	Rejeite o input ou ajuste o domínio; nunca converta overflow em saturação silenciosa.
Trace INPUT_RATE_LIMITED	Bucket por origem atingido	Reduza volume, corrija duplicação e investigue o adapter.
Divergência de hash	Inputs, ordem, build ou estado divergentes	Preserve frame/correlation ID, compare SHA/Merkle, exporte traces e reproduza.

9. Integração de rede e adapters

O Host SDK não é um parser de datagramas hostis. O fluxo recomendado é:

TEXT

datagrama externo
↓
tamanho/magic/versão/flags
↓
janela temporal e autenticação
↓
capacidade, rate limiting e anti-replay
↓
validação de comandos
↓
OperationalRuntime / adapter
↓
Host SDK em thread determinística

O `network_security` documenta, por origem autenticada, token bucket de 240 pacotes/s com burst 480, payload limitado, anti-replay e tabela limitada de origens. Esses limites não constituem proteção DDoS de borda ou rate limiting distribuído.

Um adapter pode usar fila, ring buffer ou I/O assíncrono externamente, mas deve preservar a ordem e a serialização exigidas pelo núcleo. A ABI não oferece API assíncrona genérica nem sincronização implícita.

10. Obter estado, corpos e traces

Para copiar corpos ou traces, use o padrão de duas chamadas:

C

```
uint64_t required = 0;
neoeng_dcore_status status = neoeng_dcore_host_copy_traces(
    host, NULL, 0U, &required);

/* Aloque required elementos e repita a chamada. */
```

O mesmo padrão se aplica a `neoeng_dcore_host_copy_bodies()`. As cópias retornadas pertencem ao caller; não guarde ponteiros internos do núcleo.

11. Diagnóstico de divergências

Quando duas execuções diferirem:

1. registre o commit e a identidade dos binários; 2. compare o frame e o `correlation_id`; 3. compare `stable_hash`, SHA-256 canônico e Merkle root; 4. copie os traces antes que a capacidade seja sobrescrita; 5. preserve inputs, snapshot e configuração; 6. reproduza a execução com os mesmos inputs; 7. use o diff semântico para localizar o componente conhecido; 8. corrija o adapter, input ou regra de negócio responsável; 9. repita a comparação e armazene a evidência.

Um hash divergente identifica uma diferença observada. Ele não determina sozinho a causa raiz.

12. Support bundles e evidências

Um support bundle v1 deve conter, no mínimo:

- `metadata.json`;
- `traces.json`;
- `evidence-chain.json`;
- `deferred-validation-gates.json`;
- `redaction-report.json`;
- `manifest.json`;
- `manifest.sha256`.

Não inclua chaves de sessão, segredos, material privado de assinatura ou identificadores externos brutos. Time-travel só deve ser exportado quando explicitamente autorizado.

Verifique um bundle com o verificador independente:

POWERSHELL

```
PS C:/NeoEng> python .\scripts\verify_support_bundle.py .\evidence\support-case-001
```

O verificador rejeita caminhos inseguros, symlinks, arquivos extras, entradas ausentes, tamanho divergente, SHA-256 incorreto e manifesto adulterado.

Para verificar a release consolidada:

POWERSHELL

```
PS C:/NeoEng> python .\scripts\verify_consolidated_release.py .\build\release-1.14.1
```

Para repetir as verificações normativas do repositório:

POWERSHELL

```
PS C:/NeoEng> python .\scripts\generate_manifest.py --check
PS C:/NeoEng> python .\scripts\verify_product_contract.py --check-report
PS C:/NeoEng> python .\scripts\verify_product_assurance.py
PS C:/NeoEng> python .\scripts\verify_release_assurance.py
PS C:/NeoEng> python .\scripts\verify_final_acceptance.py --check-report
PS C:/NeoEng> python .\scripts\verify_isolation.py
PS C:/NeoEng> python .\scripts\verify_host_sdk_boundary.py
```

13. Capturas reais de referência

Benchmark local — 10.000 corpos / 300 frames

```
{
  "body_count": 10000,
  "measured_frames": 300,
  "frame_p50_ms": 0.170500,
  "frame_p95_ms": 0.202000,
  "frame_p99_ms": 0.314400,
  "rollback_frames": 8,
  "rollback_ms": 0.972500,
  "final_hash": "0xE52501D04255215A"
}
```

Captura real 1: Benchmark local — 10.000 corpos / 300 frames. Saída bruta local: benchmark.stdout.txt.

Probe de determinismo — estratégias comparadas

```
full_copy=0x0FBBFF1EDC7B9346
delta_checkpoint=0x0FBBFF1EDC7B9346
paged_cow=0x0FBBFF1EDC7B9346
persistent_chunk_tree=0x0FBBFF1EDC7B9346
component_soa=0x0FBBFF1EDC7B9346
hybrid_adaptive=0x0FBBFF1EDC7B9346
immutable_chunked_canonical=0x0FBBFF1EDC7B9346
immutable_chunked_merkle=0x8EC781D8808091D0
active_binary_canonical=0x0FBBFF1EDC7B9346
active_component_soa_canonical=0x0FBBFF1EDC7B9346
active_radix16_canonical=0x0FBBFF1EDC7B9346
active_radix32_canonical=0x0FBBFF1EDC7B9346
active_radix64_canonical=0x0FBBFF1EDC7B9346
```

Captura real 2: Probe de determinismo — estratégias comparadas. Saída bruta local: determinism.stdout.txt.


```
State evidence — SHA-256, Merkle e cadeia
schema=neoeng.dcore.state-evidence-chain.v1
canonical_sha256=4f0423c219fcc6f43aa6aaeaa38617114fc8b73abb291b1b4e120c2a981e1730
merkle_root_sha256=1d0684929f913733a318d9349890730a53b8a3642fc5c895c0779acd9f98b342
envelope_sha256=b9698b62b74f5f1c9abf96f2f30e5c3d4614e843df4b8297ac540f38494ebc94
branch_parent_sha256=76d938e3d4d5b808a34f7f5c158b4b5d2592f5065f2b1f94482d113756d31f19
branch_envelope_sha256=07853d7be366af048776b3c4137ae4a6db2c2225c3e35d73e767d639148333a7
proof=accepted
chain=accepted
branch=accepted
```

Captura real 3: State evidence — SHA-256, Merkle e cadeia. Saída bruta local: state-evidence.stdout.txt.

```
Support bundle — manifesto e verificação
schema=neoeng.dcore.support-bundle.v1
entries=6
manifest_sha256=c26b1f48e33478eb0f4922e5f7096c6dd6c1b484bbc7d0ffce356048c8caa2ee
deferred_gates=2
verification=none
```

Captura real 4: Support bundle — manifesto e verificação. Saída bruta local: support-bundle.stdout.txt.

```
Replay — 128 ticks e checkpoint
replay_ticks=128
checkpoint_count=1
final_hash=0x8C484B6B83EB874C
schema_version=1
legacy_migration=passed
passed=true
```

Captura real 5: Replay — 128 ticks e checkpoint. Saída bruta local: preclosure-replay.stdout.txt.

Abaixo estão trechos de resultados realmente registrados, não valores inventados:

TEXT

```
100% tests passed, 0 tests failed out of 89
```

Esse trecho está em

docs/changesets/015/evidence/windows-x86_64-clang-20260810/raw/ctest-output.txt e representa a suíte local Windows registrada para a baseline publicada.

O relatório final de aceitação contém:

JSON

```
{
  "acceptance_state": "accepted",
  "open_internal_limitation_ids": [],
  "open_internal_requirement_ids": [],
  "status": "passed"
}
```

Isso comprova a aceitação do escopo horizontal declarado. Não comprova ARM64, certificação externa, P1 ou desempenho universal.

14. View Lab

O View Lab é um companion opcional e somente leitura:

POWERSHELL

```
PS C:/NeoEng> .\scripts\windows\run-view-lab.ps1 -BootstrapDependencies
```

Ele pode gerar traces, imagens BMP, HTML e correlação visual. Esses artefatos servem para inspeção e diagnóstico; não substituem o estado canônico nem dão autoridade de mutação ao renderer.

15. Como relatar uma execução profissionalmente

Sempre registre:

TEXT

produto: NeoEng D-Core
versão/tag:
commit:
data/hora:
sistema operacional/build:
arquitetura:
compilador/versão:
CMake/Ninja:
presença de hypervisor:
CPU/RAM/GPU/armazenamento:
política de energia/clocks/termal:
configuração CMake:
corpus/workload:
comando exato:
resultado bruto:
hash dos binários:
manifesto/evidências:
limitações aplicáveis:

Uma medição deve ser descrita como resultado daquele ambiente. Não escreva “o D-Core garante X” quando a evidência apenas mostra “X foi observado neste PC e nesta configuração”.

16. O que não deve ser prometido

Não transforme este guia em promessa de:

- 1.000 ticks/s em qualquer máquina;
- equivalência automática em Intel, AMD e ARM;
- zero alocação sem campanha P1 apropriada;
- proteção DDoS completa;
- certificação numérica global;
- certificação ISO, SOC, IEC ou DO-178C;
- prontidão mission-critical irrestrita;
- adapters setoriais prontos;
- ausência de todos os bugs;
- correção de efeitos externos irreversíveis.

Essas limitações são parte do produto e devem acompanhar qualquer proposta comercial ou relatório de auditoria.

17. Documentos normativos relacionados

- docs/contracts/HOST_SDK_C_ABI_V1.md
- docs/contracts/TEMPORAL_CLOSURE_V1.md
- docs/contracts/STATE_EVIDENCE_V1.md
- docs/contracts/SUPPORT_BUNDLE_V1.md
- docs/contracts/HARDWARE_QUALIFICATION_V2.md
- docs/architecture/HOST_SDK_BOUNDARY.md
- docs/architecture/PRODUCTION_SECURITY_BOUNDARY.md
- docs/commercial/PUBLIC_CLAIMS.md

- docs/governance/NEOENG_DCORE_SOURCE_OF_TRUTH.md
- audit/PRODUCT_CLAIMS_LEDGER.json
- audit/DEFERRED_VALIDATION_GATES.json

Em caso de conflito, os contratos e ledgers normativos prevalecem sobre exemplos, README, apresentações ou material comercial.